Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Arquitectura de Computadores y Ensambladores 2 A
Ing. Gabriel Alejandro Díaz
Auxiliar 1: Josue Antonio Cux Barrientos
Auxiliar 2: Marcos Geovani Josías Pérez Secay

Título del Proyecto:

SmartEgg: Incubadora Inteligente IoT Fase 3

PONDERACIÓN: 40 pts

Tiempo estimado: 30 hrs

Índice

1. MARCO FORMATIVO **3**
1.1. Valor 3
1.2. Competencia(s) 3
1.3. Habilidad(es) blandas a formar 3
2. Resultado del Aprendizaje **4**
2.1. Objetivos 4
3. Resumen Ejecutivo **5**
4. Enunciado del Proyecto **5**
4.1 Descripción del problema o necesidad a resolver 5
1. Integración de Servicios Cloud 6
2. Configuración de broker MQTT 6
3. Integración de Machine Learning 7
4. Sistema de gestión inteligente de consumo eléctrico: 7
5. Implementación de Dashboards de Monitoreo 8
6. Pitch y presentación final 8
7. Documentación 8
8. Repositorio de GitHub 9
9. Restricciones: 9
4.2 Alcance del proyecto
Alcance obligatorio: 9
4.3 Recursos y herramientas a utilizar 10
4.4 Entregables 12
5. Material de apoyo 13
6. Metodología **14**
Bloque A: Modelo de Machine Learning para Detección de Anomalías 14
Paso 1: Extracción y exploración del dataset histórico 14
Paso 2: Preprocesamiento y etiquetado de datos 14
Paso 3: Entrenamiento y evaluación del modelo 14
Paso 4: Integración del modelo al pipeline MQTT/backend 14
Bloque B: Visión Computacional con OpenCV 15
Paso 5: Integración de la cámara al sistema físico 15
Paso 6: Implementación del módulo de análisis visual con OpenCV 15
Bloque C: Migración a la Nube y Dashboards de Grafana 15
Paso 7: Preparación y migración del stack Docker a la nube 15
Paso 8: Configuración de dashboards y alertas en Grafana 15
Paso 9: Pruebas de integración completa y documentación final 15
7. Cronograma **16**
8. Rúbrica de Calificación **16**
8.1 Requisitos para optar a la calificación 16

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Innovación y Pensamiento Computacional	Se impulsa la integración de inteligencia artificial, visión computacional y despliegue en la nube sobre una infraestructura IoT existente, fomentando la creatividad para transformar datos históricos en capacidades predictivas y de toma de decisiones autónoma.

1.2. Competencia(s)

Competencia General:

- Aplica principios avanzados de ingeniería de sistemas, ciencias de la computación e inteligencia artificial en la construcción de soluciones complejas e integradas sobre infraestructuras IoT existentes.

Competencia Específica:

- Diseña e implementa pipelines de machine learning y visión computacional integrados a sistemas de tiempo real, evaluando su desempeño con métricas cuantitativas y desplegando los resultados en entornos de nube con capacidades de monitoreo y alerta.

1.3. Habilidad(es) blandas a formar

Comunicación efectiva:

Presentación clara de decisiones técnicas complejas: justificación de la elección del modelo de ML, interpretación de métricas de desempeño y explicación de los resultados de visión computacional ante audiencias técnicas y no técnicas.

Resolución de problemas:

Identificación y corrección de fallos en el pipeline de datos, ajuste de hiperparámetros del modelo, manejo de errores de captura de imagen y depuración de configuraciones de despliegue en la nube.

Pensamiento crítico:

Evaluación objetiva del desempeño del modelo de anomalías, decisión sobre umbrales de alerta, análisis crítico de las limitaciones de la visión computacional en contextos de incubadora y comparación de alternativas de infraestructura en la nube.

Adaptabilidad:

Capacidad de extender y migrar una arquitectura existente hacia nuevas capas de inteligencia y nube, gestionando la incertidumbre técnica propia de proyectos de integración a gran escala.

2. Resultado del Aprendizaje

2.1. Objetivos

Específico (¿Qué?)	Ampliar el sistema Smart Egg de la Fase 2 incorporando tres capas de inteligencia: (1) un modelo de machine learning para detección de anomalías en datos de sensores, (2) visión computacional con OpenCV para análisis visual del interior de la incubadora y (3) migración del stack Docker completo a un proveedor de nube con dashboards Grafana y alertas automáticas.
Medible (¿Cuánto?)	El proyecto se verificará mediante: precisión del modelo de anomalías evaluada con al menos dos métricas (ej. F1-score, matriz de confusión); detección funcional de al menos una condición visual con OpenCV; despliegue exitoso en nube con Grafana mostrando datos en tiempo real y al menos una regla de alerta activa.
Alcanzable (¿Cómo?)	Se utilizarán los datos históricos del Proyecto 2 para entrenar el modelo ML, una cámara adicional para la visión computacional, bibliotecas Python (scikit-learn, OpenCV), y un proveedor de nube accesible (AWS, GCP, Azure o equivalente) que soporte Docker y Grafana.
Realista (¿Para qué?)	El proyecto busca demostrar cómo una infraestructura IoT puede evolucionar de la simple telemetría hacia la inteligencia operativa, combinando análisis predictivo, percepción visual y acceso remoto en la nube para una supervisión autónoma y confiable.
A Tiempo (¿Cuándo?)	El proyecto deberá completarse en el período establecido del laboratorio, incluyendo las fases de entrenamiento del modelo, integración de cámara, migración a la nube, configuración de Grafana y presentación final con demostración en vivo.

3. Resumen Ejecutivo

La Fase 3 del proyecto Smart Egg extiende la incubadora inteligente IoT construida en la Fase 2, incorporando tres capas de inteligencia sobre la infraestructura ya existente.

En primer lugar, se utilizarán los datos históricos de todos los sensores Fase 1 y Fase 2 donde servirán para entrenar un modelo de machine learning orientado a la detección de anomalías.

El modelo será evaluado con métricas de desempeño y se integrará al pipeline MQTT/backend existente para emitir alertas automáticas cuando el comportamiento del sistema se desvíe de los rangos operativos esperados.

En segundo lugar, se añadirá visión por computadora con IA (OpenCV + modelo entrenado) que verifica que lo que hay dentro de la incubadora es realmente un huevo. Si el operador inicia el conteo/ciclo pero la cámara **detecta un objeto que no es un huevo, suena la alarma y todo el sistema se bloquea** incluyendo la página y el sistema.

En tercer lugar, la migración a la nube (AWS o Azure): máquina virtual Linux con Docker, base de datos (en Docker o servicio gestionado), broker MQTT, Grafana y una página web alojada en un bucket (S3 / Azure Blob Static Website) que se conecta a la VM.

El resultado es la misma incubadora SmartEgg, ahora dotada de análisis predictivo, percepción visual inteligente y presencia en la nube.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Aunque la Fase 2 logró monitorear y controlar variables ambientales en tiempo real, el sistema resultante carece de capacidades predictivas: sólo reacciona a condiciones fuera de rango después de que estas ya ocurrieron, sin aprender del comportamiento histórico ni detectar patrones sutiles que preceden a fallas. Además, la supervisión es exclusivamente numérica —sensores de temperatura y humedad— sin ninguna percepción visual del estado físico del interior de la incubadora. Finalmente, la arquitectura sigue siendo local, lo que limita el acceso remoto y la escalabilidad.

La Fase 3 cierra tres brechas: análisis predictivo, percepción visual con seguridad y acceso remoto en la nube.

Deberá cumplir con los siguientes requisitos técnicos esenciales:

1. Integración de Servicios Cloud

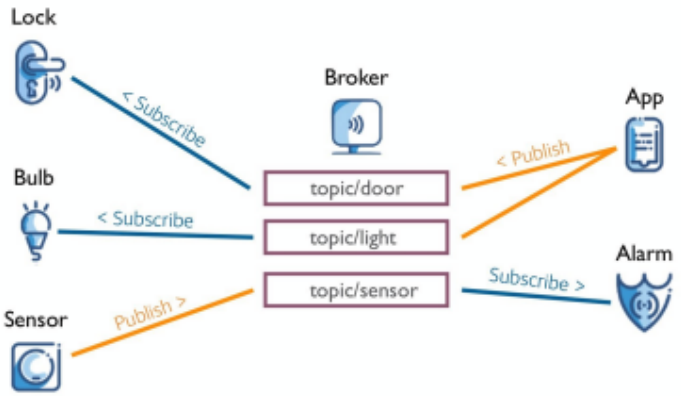
Implementación de una arquitectura en la nube que incluya:

- Elastic Computing Cloud (EC2): Despliegue de servidor IaaS para gestionar la lógica central del sistema domótico. (**obligatorio**)
- Amazon Relational Database (RDS): Utilización de base de datos relacional PaaS para almacenamiento estructurado de información de dispositivos, usuarios y configuraciones. (Es **obligatorio**, pero si el caso que la base este levantándose con la **docker en la EC2** estará permitido)
- Simple Storage Service (S3): Despliegue del website (FrontEnd) en Internet y almacenamiento de objetos como imágenes, vídeos, entre otros. Este es opcional



2. Configuración de broker MQTT

Implementación exclusiva de protocolo MQTT para la comunicación entre dispositivos IoT y el sistema central, eliminando APIs intermediarias. El broker deberá estar configurado en Internet y será el único medio para controlar sensores y actuadores del sistema.



3. Integración de Machine Learning

Incorporación de al menos una de las siguientes capacidades de aprendizaje automático:

- **Computer Visión IA (huevo vs. objeto)**: Sistemas de reconocimiento visual, modelo entrenado por el equipo con dataset propio (fotos de huevos, objetos distintos y cámara vacía, con distintas iluminaciones). Clases mínimas: `huevo`, `objeto extraño`, `vacío`.
- **OpenCV para captura y preprocesamiento**; el modelo clasifica cada fotograma (las imágenes tienen que ser reales y tiene que ver evidencias).

4. Sistema de gestión inteligente de consumo eléctrico:

- Implementación de un sistema que monitoree **el consumo eléctrico en tiempo real**, puede tener capacidad de programar horarios de funcionamiento de dispositivos, establecer límites de consumo y un temporizador de conexión eléctrica. Deberá incluir tablero de visualización y alertas automatizadas cuando se detecten consumos anómalos. Para esta función se debe de justificar los cálculos de consumo con valores aproximados a los reales, y que sean coherentes.

5. Implementación de Dashboards de Monitoreo

El sistema deberá contar con dos interfaces monitoreo que cumplan con los siguientes requisitos:

- Dashboard con Grafana: Implementación de un panel de control utilizando Grafana para la visualización avanzada de métricas en tiempo real.
- Dashboard con Processing o P5.js: Desarrollo de una interfaz visual interactiva utilizando Processing (Java/Python) o P5.js (JavaScript).

6. Pitch y presentación final

Cada equipo deberá exponer un pitch de un máximo de 5 minutos ante el panel evaluador, integrado por el catedrático y los auxiliares del curso, simulando la presentación de una propuesta ante un fondo de innovación.

La exposición deberá presentar de forma clara y ordenada el problema abordado, la solución SmartEgg y su arquitectura (modelo de anomalías, visión por computadora con IA, despliegue en la nube y dashboards), y destacar el valor disruptivo del bloqueo inteligente por detección visual.

El pitch deberá incluir una demostración en vivo con el sistema funcionando en tiempo real, incluyendo la reacción del sistema (alarma y bloqueo) ante la presencia de un objeto que no sea un huevo. Al finalizar, el panel realizará una ronda de preguntas técnicas para evaluar el dominio del equipo sobre las decisiones de diseño, el modelo entrenado, la infraestructura en la nube y el código implementado. Se evaluará la claridad y estructura de la narrativa (3 pts), el dominio técnico (3 pts), la capacidad de persuasión y propuesta de valor (2 pts) y el cumplimiento del tiempo (2 pts).

No presentar el pitch o exceder el tiempo máximo sin autorización previa del panel penalizará el criterio correspondiente con un 20 %.

7. Documentación

La documentación deberá de realizarse en formato Markdown en el README del repositorio de la práctica, dicho lo anterior se solicita:

Manual Técnico:

- Descripción de la Solución
- Capas del Framework IoT
- Documentación de las Bases de Datos (Modelo Entidad - Relación)
- Documentación API (Debe ser generada por Postman, Swagger o alguna herramienta similar)
- Scripts para cada consulta hecha a la base de datos durante la calificación
- Explicación y justificación de uso del Valor Disruptivo.

Manual de Usuario:

- Uso e interpretación de los dashboards
- Uso del Panel de Administración/Aplicación.
- Uso de los botones y paneles para accionar los actuadores.
- Uso de cualquier funcionalidad extra como Valor Disruptivo.

8. Repositorio de GitHub

Todo el código utilizado y la documentación deberá ser subido a un repositorio de GitHub. En la entrega sólo deberá contener el link del repositorio. Se utilizará el mismo repositorio, solo se deberá de almacenar en la nueva Carpeta de Proyecto 2.

9. Restricciones:

- Utilizar API: Queda prohibido el uso de API, se debe usar el broker MQTT únicamente. (-100%)
- Cloud Deploy: Si no se encuentran el broker MQTT y la base de datos en la nube no se realizará la calificación. (-100%)
- Sin uso de Grafana/Processing/p5.js: Se calificará únicamente a través de dashboards con estas tecnologías.
- Entrega Tarde: Se deberán de mandar todos los entregables y commits en la fecha establecida, de no ser así se tendrá una penalización del 25%.
- Documentación: Si no se cuenta con la documentación de determinada sección, no se podrá optar a calificación.
- Sin módulo de visión funcional: −100 %.
- No presentar el pitch o exceder 5 min sin autorización: −20 % en ese criterio.
- Entrega tardía: −25 %.
- Falta de documentación de una sección obligatoria: no se califica esa sección

4.2 Alcance del proyecto

Alcance obligatorio:

- Procesamiento y preparación del dataset generado en la Fase 2.

Hardware y maqueta

- Incubadora física de la Fase 2 funcionando en vivo (ESP32/Arduino, sensor DHT, actuadores).
- Actuadores de al menos 3 categorías (ej. calefacción/foco, ventilación, volteo de huevos, señalización/buzzer).
- Buzzer/alarma física y señal luminosa para el estado de bloqueo.
- Cámara (ESP32-CAM, USB o Raspberry Pi Camera) apuntando al interior.

Visión IA (huevo vs. objeto)

- Modelo entrenado por el equipo con dataset propio (fotos de huevos, objetos distintos y cámara vacía, con distintas iluminaciones). Clases mínimas: `huevo`, `objeto extraño`, `vacío`.
- OpenCV para captura y preprocesamiento; el modelo clasifica cada fotograma (o región de interés).
- Imágenes reales, no simuladas.

Nube (AWS o Azure)

- Máquina virtual Linux (EC2 con Ubuntu / VM de Azure con Ubuntu) con Docker y Docker Compose ejecutando: broker MQTT, backend, módulo ML, módulo de visión y Grafana.
- Base de datos relacional, con una de estas dos opciones (justificar la elegida):
  - a) Contenedor Docker en la misma VM (PostgreSQL/MySQL) con volumen persistente.
  - b) Servicio gestionado (RDS / Azure Database).
- Bucket (S3 o Azure Blob Storage) con hosting estático para la página web/dashboard interactivo (p5.js o Processing.js), que se conecta a la VM por su IP pública.
- Grupos de seguridad/firewall abriendo solo los puertos necesarios.

Comunicación

- MQTT como único canal de control de sensores y actuadores. Prohibido usar APIs REST para controlarlos.

Dashboards

- Grafana (tiempo real e histórico): sensores, actuadores, salida del modelo ML, resultado de visión y estado del sistema (normal / bloqueado).
- Página web en bucket (p5.js/Processing) conectada al broker (MQTT sobre WebSockets) o a la base de datos.
- Al menos una alerta activa en Grafana (umbral o anomalía ML).

Presentación

- Pitch de máximo 5 minutos, demo en vivo y ronda de preguntas técnicas.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Hardware	Sistema físico de incubadora completo proveniente de la Fase 2 (Arduino/ESP32, sensores DHT, actuadores).
Obligatorio	Hardware	Cámara compatible con el microcontrolador o con el servidor backend (cámara USB, ESP32-CAM, Raspberry Pi Camera u otra equivalente).

Obligatorio	Software / Biblioteca	Python con scikit-learn (u otra biblioteca de ML equivalente) para entrenamiento y evaluación del modelo de detección de anomalías.
Obligatorio	Software / Biblioteca	OpenCV (cv2) para procesamiento de imágenes y visión computacional en el módulo de análisis visual.
Obligatorio	Plataforma / Nube	Proveedor de nube (AWS, GCP, Azure, DigitalOcean u equivalente) con soporte para instancias con Docker.
Obligatorio	Plataforma	Grafana para visualización de dashboards y configuración de alertas sobre los datos del sistema.
Obligatorio	Plataforma	Docker y Docker Compose para contenerización y migración del stack completo a la nube.
Obligatorio	Plataforma	GitHub para control de versiones, documentación y gestión del repositorio del proyecto.
Obligatorio	Plataforma	Broker MQTT Mosquitto (o equivalente en la nube) para comunicación IoT en el entorno migrado.
Opcional	Software / Biblioteca	Pandas y NumPy para exploración y preprocesamiento del dataset histórico de sensores.
Opcional	Software / Biblioteca	Matplotlib o Seaborn para visualización del análisis exploratorio y métricas del modelo.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Documento técnico	Documento que incluya los objetivos de la Fase 3, arquitectura del sistema extendido, descripción del pipeline de ML (preprocesamiento, entrenamiento, evaluación), descripción del módulo de visión computacional, diagrama de la infraestructura en nube y explicación del stack de monitoreo con Grafana.
Manual de usuario	Documento que describa el uso del sistema extendido: cómo interpretar las alertas del modelo de ML, cómo acceder a los dashboards de Grafana en la nube, cómo funciona el módulo de visión computacional y consideraciones generales de operación.
Notebook de ML	Jupyter Notebook o script Python documentado que incluya la carga y exploración del dataset, el preprocesamiento de datos, el entrenamiento del modelo de detección de anomalías, la evaluación con métricas y el guardado del modelo para su integración al backend.
Módulo de visión computacional	Código fuente documentado del módulo OpenCV, con descripción de las condiciones visuales detectadas, lógica de análisis implementada y ejemplos de imágenes procesadas.
Infraestructura en la nube	Evidencia del despliegue exitoso del stack Docker en el proveedor de nube: capturas del servicio en ejecución, URL de acceso al dashboard de Grafana y configuración de alertas activas.
Repositorio Git	Repositorio con estructura organizada: código fuente del modelo ML, módulo de visión computacional, configuración de infraestructura (docker-compose, archivos de nube), dashboards exportados de Grafana y documentación. Se deberá evidenciar trabajo progresivo mediante commits.
Evidencias de funcionamiento	Capturas, fotografías o videos que demuestren: el modelo emitiendo una alerta ante una anomalía real o simulada, OpenCV analizando imágenes de la incubadora, y los dashboards de Grafana mostrando datos en tiempo real desde la nube.

5. Material de apoyo

Categoría (Manual / Documentación oficial/	Link	Descripción

Ejemplo / Etc)		
Documentación oficial	https://scikit-learn.org/stable/	Documentación completa de scikit-learn: modelos de ML, métricas de evaluación, pipelines y preprocesamiento de datos.
Documentación oficial	https://docs.opencv.org/4.x/	Referencia oficial de OpenCV para Python: procesamiento de imágenes, detección de contornos, análisis de video y visión computacional.
Manual / Tutorial	https://docs.grafana.com/	Documentación de Grafana: creación de dashboards, configuración de fuentes de datos y alertas.
Manual / Tutorial	https://docs.docker.com/compose/	Guía oficial de Docker Compose para orquestación de servicios multi-contenedor.
Documentación oficial	https://docs.aws.amazon.com/ec2/	Documentación de Amazon EC2 para creación y gestión de instancias en la nube (puede sustituirse por GCP, Azure u otro proveedor).
Ejemplo / Tutorial	https://scikit-learn.org/stable/modules/outlier_detection.html	Guía de scikit-learn para detección de valores atípicos y anomalías con métodos como Isolation Forest y Local Outlier Factor.
Manual / Tutorial	https://randomnerdtutorials.com/esp32-cam-video-streaming/	Tutorial de captura de imágenes y streaming con ESP32-CAM, útil como referencia de integración de cámara al sistema físico.
Documentación oficial	https://pandas.pydata.org/docs/	Documentación de Pandas para carga, exploración y preprocesamiento de datasets tabulares de sensores.
Ejemplo	https://github.com/	Plataforma para control de versiones, gestión del proyecto y publicación del repositorio.

6. Metodología

A continuación se presenta la metodología sugerida para elaborar el presente proyecto. Los pasos están organizados en tres bloques que corresponden a las tres capas de inteligencia a integrar:

Bloque A: Modelo de Machine Learning para Detección de Anomalías

Paso 1: Extracción y exploración del dataset histórico

Descripción:
Extraer los datos de temperatura, humedad y estado de actuadores almacenados en la base de datos del Proyecto 2. Realizar un análisis exploratorio: estadísticas descriptivas, visualización de distribuciones, identificación de valores nulos o duplicados, y detección de patrones de comportamiento normal en el sistema.

Paso 2: Preprocesamiento y etiquetado de datos

Descripción:
Limpiar el dataset, normalizar o estandarizar variables según el algoritmo elegido, y definir la estrategia de detección de anomalías (supervisada o no supervisada). Si el enfoque es no supervisado (ej. Isolation Forest, LOF), no es necesario un etiquetado manual; si es supervisado, generar o simular etiquetas de anomalía para el entrenamiento.

Paso 3: Entrenamiento y evaluación del modelo

Descripción:
Entrenar el modelo seleccionado sobre el dataset preprocesado y evaluar su desempeño mediante métricas apropiadas: matriz de confusión, precisión, recall, F1-score o AUC-ROC según el enfoque utilizado. Ajustar hiperparámetros si es necesario y documentar los resultados obtenidos.

Paso 4: Integración del modelo al pipeline MQTT/backend

Descripción:
Desarrollar la lógica de integración en el backend (según el stack definido en el Proyecto 2) que cargue el modelo entrenado, evalúe en tiempo real las lecturas recibidas vía MQTT y publique automáticamente una alerta hacia el broker cuando se detecte un comportamiento anómalo.

Bloque B: Visión Computacional con OpenCV

Paso 5: Integración de la cámara al sistema físico

Descripción:
Seleccionar e instalar la cámara en el sistema de incubadora (ESP32-CAM, cámara USB u otra). Configurar la captura de imágenes o video desde el backend, verificando resolución, frecuencia de captura y almacenamiento temporal de fotogramas. Realizar pruebas de calidad de imagen bajo las condiciones de iluminación del entorno.

Paso 6: Implementación del módulo de análisis visual con OpenCV

Descripción:
Diseñar e implementar el módulo de visión computacional que analice los fotogramas capturados. Implementar al menos una de las siguientes funcionalidades según el caso de uso elegido: detección de presencia/ausencia de objetos, análisis de cambios entre fotogramas (detección de movimiento), análisis de color o temperatura visual, o detección de contornos/formas anómalas. Los resultados del análisis deben quedar disponibles para ser publicados y visualizados en Grafana

Bloque C: Migración a la Nube y Dashboards de Grafana

Paso 7: Preparación y migración del stack Docker a la nube

Descripción:
Revisar y adaptar el docker-compose del Proyecto 2 para su despliegue en una instancia de nube. Provisionar el servidor virtual en el proveedor elegido, instalar Docker y Docker Compose, y transferir el stack. Configurar los grupos de seguridad o reglas de firewall para exponer únicamente los puertos necesarios. Verificar que todos los servicios (broker MQTT, backend, base de datos, Grafana) operen correctamente en el entorno de nube.

Paso 8: Configuración de dashboards y alertas en Grafana

Descripción:
Conectar Grafana a la fuente de datos del sistema (InfluxDB, PostgreSQL, etc.) y construir dashboards que visualicen: lecturas de temperatura y humedad en tiempo real, estado de actuadores, salida del modelo de detección de anomalías y resultados del módulo de visión computacional. Configurar al menos una alerta automática en Grafana asociada a un umbral crítico o a la detección de una anomalía por el modelo ML.

Paso 9: Pruebas de integración completa y documentación final

Descripción:
Ejecutar pruebas end-to-end del sistema completo: el nodo físico envía datos por MQTT al stack en la nube, el modelo ML los evalúa y emite alertas si detecta anomalías, la cámara captura imágenes que son analizadas por OpenCV, y todo se refleja en tiempo real en Grafana. Documentar el proceso, recopilar evidencias y preparar la presentación final.

Recomendaciones

- Comenzar con el Bloque A lo antes posible, ya que el dataset histórico puede ser pequeño al inicio y el entrenamiento del modelo requiere iteraciones.
- Probar el módulo de OpenCV de manera aislada antes de integrarlo al backend; capturar imágenes de prueba con antelación para no depender del hardware durante el desarrollo.
- Para la migración a la nube, usar la instancia más pequeña disponible que soporte Docker; no es necesario un servidor de alto rendimiento para este proyecto.
- Exportar los dashboards de Grafana como JSON y versionarlos en el repositorio para facilitar la replicación del entorno.
- Documentar cada decisión de diseño: elección del algoritmo de ML, umbral de detección, condiciones visuales analizadas y configuración de infraestructura.
- Mantener el Proyecto 2 funcionando en paralelo como referencia de comparación antes de migrar definitivamente.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Lectura	19-09-2026	23-10-2026
Entrega	23-10-2026	24-10-2026
Calificación	24-10-2026	24-10-2026

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar. Todo proyecto debe incluir estos criterios desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos indicados a continuación. De no cumplir, la nota tendrá valor de cero puntos.

Tema	**Descripción**	**Cumple (Sí/No)**

Modelo de ML funcional e integrado	El modelo de detección de anomalías debe estar entrenado, evaluado con al menos una métrica y correctamente integrado al pipeline del sistema para emitir alertas automáticas.	
Módulo de visión computacional operativo	El módulo OpenCV debe estar implementado, capturando imágenes desde la cámara y produciendo resultados de análisis visual observables en el sistema.	
Stack desplegado en la nube	El stack Docker completo debe estar en ejecución en el proveedor de nube elegido y ser accesible de manera remota durante la presentación.	
Dashboard de Grafana con alertas	Grafana debe mostrar dashboards con datos en tiempo real y contar con al menos una alerta configurada y funcional.	
Evidencias del desarrollo	Deben presentarse capturas, fotografías o videos que evidencien el funcionamiento de las tres capas implementadas.	
Repositorio del proyecto	El repositorio Git debe contener código organizado, evidencia de trabajo progresivo mediante commits y la documentación requerida.	
Documentación obligatoria	Deben entregarse el documento técnico, el notebook de ML, el manual de usuario y la bitácora de desarrollo.	